

Fiducia controls how AI agents spend money.

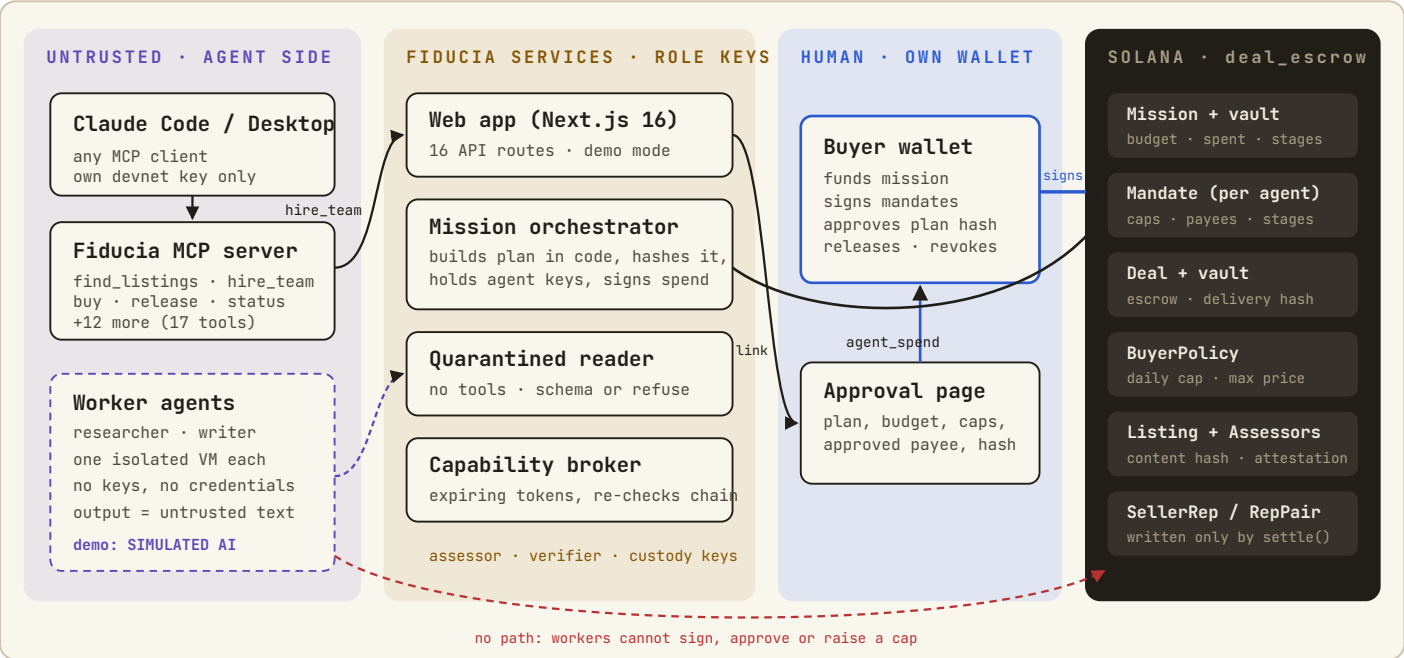
A single Solana program turns a human's budget into on-chain **mandates**: per-agent caps, per-payment caps, approved payees, stages and expiry. Agents can buy data, APIs and specialist work, but every payment is checked by the chain, every stage needs the human's signature on the exact plan hash, and the team is paid only for the product hash it delivered.

LIVE ON SOLANA DEVNET

fiducia-orpin.vercel.app
github.com/MuLaydm10/solana-hackathon

1	56	630 KB	403	17	<\$0.001
ANCHOR PROGRAM, ALL STATE IN PDAS	TYPED REFUSAL CODES	DEPLOYED BINARY, SHA256 PINNED	TEST CASES IN 5 PACKAGES	MCP TOOLS FOR AGENTS	MEDIAN SOLANA FEE, Q4 2025*

SYSTEM MAP AND TRUST BOUNDARIES



AGENT SIDE

Any MCP client calls `hire_team`. The tool returns an approval URL and **signs nothing**. Worker agents run in their own VM with no keys; they can only ask the broker for scoped, expiring capabilities. Everything they emit is treated as untrusted text.

FIDUCIA SERVICES

The orchestrator renders each stage plan **in code**, hashes it and waits for the human. It holds one keypair per agent role and signs `agent_spend`, so the on-chain mandate judges every payment. Server keys are role keys only (assessor, verifier, custody), never a user's.

SOLANA IS THE REFEREE

All money sits in program-owned vaults. Caps, payees, stage, expiry, revoke, delivery hash and settlement are checked by `deal_escrow` itself. If a server is compromised, it still cannot move funds outside the mandate the buyer signed.

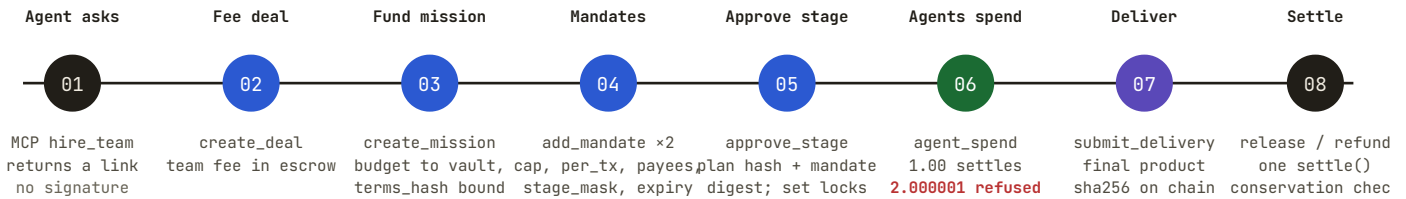
CODE MAP • MAIN @ E7E63C4

chain/ 97 tests Anchor program, 1,979 lines Rust; TS client.	core/ 66 tests Terms hashes, pricing, blueprints, reputation.	agents/ 115 tests Orchestrator, broker, reader, custody.	web/ 93 tests Next.js 16, 16 API routes, e2e.	mcp/ 32 tests 17 tools over stdio.
---	--	---	---	--

* Blockworks Research, State of Solana Q4 2025: median fee 0.000006 SOL (~\$0.0009). Test count = static count of declared test cases on main (e7e63c4).

1 / 3

One mission, *every step a transaction.*



Human signs: 02 03 04 05 08 • Orchestrator signs for an agent: 06 07 • Agent's MCP client signs: nothing in a hire.
Stages repeat 05 to 06 per stage. revoke_mandate stops an agent in one transaction; close_mission refunds the unspent vault.

WHAT AGENT_SPEND MUST PROVE, EVERY TIME

signer = mandate.agent • mission open, not expired	
mandate not revoked, not expired	MandateRevoked • Expired
current stage approved by the buyer	StageNotApproved
agent works in this stage (stage_mask)	NotThisStage
amount ≤ per_tx_cap	OverPerTxCap
spent ≤ mandate • stage • mission cap	OverMandateCap
payee in mandate.payees, or attested listing seller	PayeeNotAllowed
pay vault → payee, emit SpendEvent{receipt_hash}	

Order in code: mission, mandate, stage gate, then caps from the smallest scope outward (OverPerTxCap, OverMandateCap, OverStageCap, OverMissionBudget). All checks are conjunctive; failure aborts the transaction, so a refused payment never lands and moves nothing. agent_open_deal applies the same caps when an agent buys through escrow, and refunds never reduce spent: caps limit outflow.

ACCOUNT MODEL (ALL PDAS)

ACCOUNT	KEY FIELDS	WRITTEN BY
Mission	budget spent stages[≤8] mandates_digest mandates_locked terms_hash expires_at	buyer; spends via agents
Mandate	agent role_hash cap per_tx_cap payees[≤8] stage_mask revoked	buyer only
Deal	amount stake bond_bps deadline review_secs terms_hash delivery_hash verifier	buyer, seller, verifier
BuyerPolicy	period_budget max_price approval_threshold allowed_sellers[≤8]	buyer only
Listing	kind content_hash meta_hash report_hash assessor	seller; assessor attests
SellerRep	completed failed distinct_buyers max_pair_volume	settle() only

WHY HASHES, NOT PROSE

The buyer approves a plan rendered by code and its sha256, never model text. approve_stage binds that plan hash and the digest of the mandate set; after the first approval the mandates are locked, so nobody can swap in a larger cap or a new payee after the human said yes.

ESCROW RULES

- ◆ Delivery hash must match the listed content (data) or the mission product (teams).
- ◆ Review window, then release or challenge with a bond.
- ◆ Independent verifier rules; no ruling = NoVerdict path.
- ◆ timeout_refund if nothing is delivered.

MARKETPLACE PRIMITIVES

- ◆ Seller chain: classify, grade A to D, price, draft terms.
- ◆ PII and secret scan; any secret = refused listing.
- ◆ Data encrypted AES-256-GCM; key sealed to buyer.
- ◆ x402 pay-per-call services: no answer, no charge.

REPUTATION THAT RESISTS WASH TRADING

- ◆ No score below 10 deals or 3 distinct buyers.
- ◆ Flag when one buyer exceeds 50% of volume.
- ◆ Updated only inside settle(), so every change comes with a real payout.

Threats we *designed against*.

THREAT	CONTROL
Agent overspends or loops	Mandate cap, per-payment cap, stage cap and mission budget on chain
Prompt injection in a listing or web page	Quarantined reader with no tools; strict schema; numbers computed in code; signals logged
Approval replay or plan swap	Approval bound to plan hash + mandates digest; mandates lock after first approval
Compromised Fiducia server	Holds role keys only; funds sit in program vaults; can never exceed the signed mandate
Credential leak from a worker	Broker capabilities, never secrets; egress allowlist; replies containing the credential are refused
Seller ships different data	Delivery must equal on-chain content hash; buyer verifies after decrypting
Fake grade or fake reputation	Attestations only from the assessor registry; reputation written only by settle()
Demo key abuse	Devnet only; small on-chain policy; per-IP and daily limits; acts only on its own missions; fails closed

Proof on *Solana devnet*.

STEP	INSTRUCTION	SIGNATURE
Fee deal opened	create_deal	5BsEfnAB...DGW5SCR7
Mission funded	create_mission	3zJQP1St...4JpMgEn
Researcher mandate	add_mandate	3xZXMDCA...914tjssA
Stage 1 approved	approve_stage	5Nhc8QQV...mNNH2o
2.000001 USDC	agent_spend	OVERPERTXCAP · NEVER LANDS
1.00 USDC to seller	agent_spend	2jLv1HdB...6HvjtmZ
Stage 2 approved	approve_stage	5Q8u4zSE...tUzEP4ix
Product hash	submit_delivery	3swDEf2k...T6K6ApUU
Team fee released	release	uKK7ggQs...ngubasE4

Mission BJyJL2qc...1rSs, researcher mandate 3.00 USDC total, 2.00 per payment, 1 payee. The demo video run (mission 3iHW4eyY...) repeated the flow; its release 5f79BRnk... returned `err: null`. Deployed ELF sha256 d4375512...7e8c is checked byte for byte by `verify-deployed.ts`.

STATUS TODAY · STATED PLAINLY	
Program, mandates, stages, escrow, refusal, settlement	LIVE DEVNET
Web app, read-only and one-click demo missions	LIVE
MCP server, 17 tools incl. hire_team	WORKING
Sell, buy data, sealed key, x402 pay-per-call	DEVNET
Worker agents in the demo	SIMULATED AI
Claude provider for workers	BUILT, NOT LIVE
Verifier rulings wired to the site	NEXT
Mainnet, real USDC, external audit	NOT YET

KNOWN LIMITS WE WILL CLOSE
◆ Upgrade authority is held by the team; it is needed to manage the assessor registry. Plan: multisig, then a timelocked upgrade policy before mainnet.
◆ The assessor sees plaintext data. Plan: sample-only assessment, then independent assessors, then TEE.
◆ Demo rate limits and broker approval nonces live in memory per instance.
◆ The orchestrator holds agent keys, so it is trusted for liveness, not for limits.
◆ Program not yet externally audited.

ENGINEERING ROADMAP

NOW · OCT 2026	NEXT · 30 DAYS	90 DAYS	MAINNET GATE
Devnet MVP Mandates, stage approvals, escrow, refusal codes, MCP hire_team, judge demo, verified binary.	Live agents Claude in the researcher and writer slots behind the same reader and mandates; provider mode reported by MCP; per-client spend export.	Hardening External audit, multisig upgrade authority, persistent limits, mandate templates for research procurement, SDK.	Capped mainnet Real USDC with low per-mission ceilings, monitored payees, incident runbook, then raise caps per partner.

Let agents spend. *Keep the control.*

Program CfD43mq2...NGSV · Solana devnet
fiducia-orpin.vercel.app · github.com/MuLaydm10/solana-hackathon